

Compilador C--

Entrega 1: Analisador Léxico

João Victor Domingos e Souza, Nicole Garcia Montes Clemente,
Marcelo Americo da Silva, Davi Marques de Oliveira

14 de setembro de 2026

1 Visão geral (Introdução)

O projeto desenvolve um compilador C-- em quatro entregas. Esta primeira parte documenta a análise léxica; as próximas acrescentarão sintaxe, semântica e simulação ao mesmo documento. A organização segue os tópicos do enunciado [Disciplina de Compiladores s.d., seção 1.1, págs. 1--2].

O programa recebe um arquivo `.cmm`, identifica tokens até EOF, consulta cada lexema válido em uma única tabela de símbolos e acumula diagnósticos com linha e coluna. A execução, a partir da raiz do projeto, é:

```
python main.py programa.cmm
```

São impressos nomes de tokens em `stdout`, um por linha, e erros em `stderr`. O retorno é 0 sem erros, 1 com erros léxicos e 2 nas falhas de entrada tratadas. A extensão `.cmm` é uma convenção do grupo.

A implementação usa Python 3.10 ou posterior, um AFD configurado em JSON e uma fachada `Lexer` que monta os componentes. A separação entre leitura, reconhecimento, armazenamento e apresentação mantém as responsabilidades dos componentes bem definidas e reduz o acoplamento entre eles. As interfaces entre fases se apoiam em [Appel e Ginsburg 1998, seção 1.1, págs. 4--5] e a responsabilidade do scanner em [Aho et al. 2007, seção 3.1.1, págs. 110--111]. O texto inteiro é carregado em memória para permitir retrocesso com posição, linha e coluna; isso exige memória proporcional à entrada. `str`, listas e dicionários evitam limites fixos de comprimento.

A gramática C-- prevalece sobre C. A especificação N3220 do WG14, será usada para comparação das regras léxicas e ao longo do restante do desenvolvimento, e não como especificação integral da linguagem do trabalho [ISO/IEC JTC 1/SC 22/WG14 2024, seção 6.4, pág. 52].

2 Analisador léxico

2.1 Tokens e Expressões Regulares

Tokens são categorias, padrões descrevem as sequências aceitas e lexemas são os textos encontrados na entrada, conforme [Aho et al. 2007, seção 3.1.2, pág. 111]. A transformação da sequência de caracteres em tokens é apresentada por Appel e Ginsburg [Appel e Ginsburg 1998, cap. 2, págs. 16--38].

Neste trabalho, os tokens foram definidos a partir da gramática da linguagem C-- e das especificações fornecidas para o analisador. O reconhecimento é realizado através de um Autômato Finito Determinístico (AFD), cuja configuração é armazenada no arquivo `AFD.json` e carregada pela classe `AFD`.

A implementação utiliza expressões regulares somente para verificar algumas classes de caracteres durante as transições. A estrutura geral do reconhecimento continua sendo definida pelos estados e transições do AFD.

2.1.1 Padrões léxicos reconhecidos

Na implementação atual, as principais classes de lexemas reconhecidas são apresentadas na Tabela 1.

Tabela 1: Principais padrões léxicos reconhecidos pelo AFD.

Classe	Padrão ou regra	Exemplo	Estado final
Identificador	<code>[A-Za-z][A-Za-z0-9]*</code>	<code>x1</code>	<code>qID</code>
Número inteiro	<code>[0-9]+</code>	<code>32</code>	<code>qNUMINT</code>
Número decimal	<code>[0-9]+[.][0-9]+</code>	<code>12.50</code>	<code>qNUMFLOAT</code>
String	Texto entre aspas duplas	<code>"texto"</code>	<code>qSTRING_FINAL</code>
Caractere	Caractere ASCII entre aspas simples	<code>'a'</code>	<code>qCHAR_FINAL</code>

2.1.2 Identificadores

O padrão distingue maiúsculas de minúsculas e adapta [Appel e Ginsburg 1998, seção 2.2, Fig. 2.2, pág. 20], acrescentando maiúsculas. A letra inicial separa esse ramo do reconhecimento de números.

O reconhecimento de um identificador começa no estado inicial `q0`. Quando uma letra é encontrada, o AFD realiza uma transição para o estado `qID`.

A partir desse estado, novas letras ou dígitos podem continuar sendo consumidos:

```
q0 -- letra --> qID
qID -- letra | dígito --> qID
```

No método `_avaliarCondicao()`, as letras são reconhecidas por: `[a-zA-Z]` e a continuação do identificador utiliza: `[a-zA-Z0-9]`. São exemplos de identificadores reconhecidos:

```
x
x1
contador
variavel10
```

O estado `qID` pertence ao conjunto de estados finais do AFD. A distinção entre ID e palavra reservada é feita após consumir o lexema completo: `if` resulta em IF, enquanto `ifx` permanece ID. O procedimento e sua fundamentação estão descritos na Subseção 2.1.8.

2.1.3 Números

O AFD diferencia números inteiros de números de ponto flutuante. Quando um dígito é encontrado no estado inicial, ocorre a transição para o estado `qNUMINT`:

```
q0      -- dígito --> qNUMINT
qNUMINT -- dígito --> qNUMINT
```

Como `qNUMINT` é um estado final, são aceitos números como: 0, 10, 32, 1250.

Quando um ponto é encontrado depois da parte inteira, o autômato passa para o estado intermediário `qDOT`:

```
qNUMINT -- "." --> qDOT
```

O estado `qDOT` não é um estado final. Para concluir o reconhecimento de um número de ponto flutuante, pelo menos um dígito deve ser encontrado depois do ponto:

```
qDOT      -- dígito --> qNUMFLOAT
qNUMFLOAT -- dígito --> qNUMFLOAT
```

Dessa forma, são exemplos de números de ponto flutuante reconhecidos: 3.14, 10.5, 125.00. O ponto sozinho não é aceito como número, nem números sem dígitos após o ponto.

O predicado usa `re.fullmatch(r'[0-9]', caractere)` e admite apenas dígitos ASCII. Não há sinal, expoente ou sufixo no lexema numérico; o subconjunto reduz os estados desta entrega em relação à sintaxe completa de C [ISO/IEC JTC 1/SC 22/WG14 2024, seções 6.4.4.1--6.4.4.3, págs. 56--63]. A separação do sinal é coerente com N3220, seção 6.4.4.3, parágrafo 12, pág. 63. Os lexemas são armazenados como texto, sem conversão, opção descrita por [Aho et al. 2007, seção 3.1.3, Ex. 3.2, pág. 113].

2.1.4 Strings e caracteres

As strings começam quando uma aspa dupla é encontrada no estado inicial:

```
q0 -- " --> qSTRING
```

No estado `qSTRING`, os caracteres continuam sendo consumidos enquanto forem diferentes de uma nova aspa dupla e de uma quebra de linha. Quando a aspa dupla de fechamento é encontrada, o autômato passa para o estado final `qSTRING_FINAL`:

```
qSTRING -- " --> qSTRING_FINAL
```

Por exemplo:

```
"texto"
"ola mundo"
""
```

A string vazia é aceita pelo fechamento imediato: representa uma sequência de comprimento zero, como em C [ISO/IEC JTC 1/SC 22/WG14 2024, seção 6.4.5, parágrafos 1 e 3, pág. 67].

O reconhecimento de caracteres utiliza uma sequência diferente:

```
q0      -- ' --> qCHAR1
qCHAR1 -- ASCII válido --> qCHAR2
qCHAR2 -- ' --> qCHAR_FINAL
```

Um exemplo é:

'a'

Na implementação atual, a condição ASCII válido é verificada através de:

```
len(caractere) == 1 and caractere.isascii()
and caractere not in ("'", "\n", "\r")
```

Assim, o conteúdo deve ser uma única unidade ASCII, diferente de aspa simples, LF (*Line Feed*) e CR (*Carriage Return*). O espaço entre aspas simples é aceito; constante vazia e três aspas simples são rejeitadas. A gramática exige uma unidade ASCII; em C, o conteúdo da constante também não é opcional e exclui aspa bruta e quebra de linha [ISO/IEC JTC 1/SC 22/WG14 2024, seção 6.4.4.5, parágrafos 1--3, págs. 63--64]. A restrição local a uma unidade difere das constantes multicaractere admitidas por C, cujo valor depende da implementação [ISO/IEC JTC 1/SC 22/WG14 2024, seção 6.4.4.5, parágrafo 11, pág. 65].

A barra invertida é conteúdo comum quando o caminho permite. '\n' é rejeitado por conter dois caracteres; "\n" preserva barra e letra. Não interpretar escapes é uma simplificação no projeto, que não afeta a análise léxica. A gramática de C-- não define escapes, portanto a implementação atual é consistente com a sintaxe da linguagem mas diferente de C [ISO/IEC JTC 1/SC 22/WG14 2024, seção 6.4.4.5, parágrafos 3--4, págs. 64--65]. Uma melhoria possível seria permitir escapes.

2.1.5 Operadores

Os operadores possuem caminhos próprios no AFD. Alguns são reconhecidos com apenas um caractere, enquanto outros dependem da leitura de um segundo caractere. A Tabela 2 apresenta os operadores implementados.

Tabela 2: Operadores reconhecidos pelo AFD.

Lexema	Estado final	Token
+	qPLUS	PLUS
++	qINC	INCREMENT
-	qMINUS	MINUS
-	qDEC	DECREMENT
*	qMULT	MULT
/	qDIV	DIV
%	qMOD	MOD
=	qASSIGN	ASSIGN
==	qEQ	EQUALS
<	qLT	LESS_THAN
<=	qLEQ	LEQ
>	qGT	GREATER_THAN
>=	qGEQ	GEQ
!	qNOT	NEG
!=	qNEQ	DIFF
&&	qAND	AND
	qOR	OR

Nos casos em que um operador pode possuir um ou dois caracteres, o primeiro estado já pode representar um lexema válido.

Por exemplo:

```
q0      -- + --> qPLUS
qPLUS  -- + --> qINC
```

Assim, + e ++ podem ser reconhecidos separadamente. A mesma ideia é utilizada para os pares -/-, =/==, </<=, >/>= e !/!=.

2.1.6 Símbolos de pontuação

Cada símbolo de pontuação possui um caminho próprio a partir de q0. Os símbolos reconhecidos são apresentados na Tabela 3.

Tabela 3: Símbolos de pontuação reconhecidos pelo AFD.

Lexema	Estado final	Token
(	qLPARENT	LPARENT
)	qRPARENT	RPARENT
[	qLBRACKET	OPEN_BRACKETS
]	qRBRACKET	CLOSE_BRACKETS
{	qLBRACE	OPEN_BRACES
}	qRBRACE	CLOSE_BRACES
,	qCOMMA	COMMA
;	qSEMICOLON	SEMICOLON

A utilização de tokens separados para cada operador e símbolo de pontuação segue a forma recomendada nas especificações do trabalho.

2.1.7 Comentários e espaços em branco

Os comentários aceitos pela linguagem possuem o formato: `/* comentario */`. O reconhecimento começa pelo mesmo caractere utilizado pelo operador de divisão. Inicialmente, / leva o autômato ao estado qDIV. Caso o caractere seguinte seja *, o AFD segue para o caminho de comentário:

```
q0      -- / --> qDIV
qDIV    -- * --> qCOMMENT
qCOMMENT -- * --> qCOMMENTSTAR
qCOMMENTSTAR -- / --> qCOMMENTDONE
```

O estado final qCOMMENTDONE indica um comentário completo, que é descartado sem gerar token por repetição iterativa do reconhecimento. Não há aninhamento, conforme a delimitação de C [ISO/IEC JTC 1/SC 22/WG14 2024, seção 6.4.9, parágrafo 1 e nota 80, pág. 70]. // não inicia comentário em C--, embora seja permitido em C. Espaços dentro de strings são preservados. Se EOF ocorrer antes de */, o lexema completo produz “Comentário não terminado”.

Os espaços em branco também não geram tokens. No estado inicial, a condição de whitespace mantém o autômato no próprio q0:

q0 -- whitespace --> q0

Na classe **AFD**, essa condição verifica explicitamente o conjunto formado por espaço comum, tabulação horizontal, LF e CR. `Reconhecedor._ignorar_espacos()` usa o mesmo conjunto. O descarte de espaços e comentários entre tokens é uma operação comum durante a análise léxica e também é discutido por Appel e Ginsburg [Appel e Ginsburg 1998, cap. 2, págs. 16--38].

2.1.8 Palavras reservadas

As palavras reservadas não possuem estados próprios no AFD. Elas seguem inicialmente o mesmo caminho utilizado pelos identificadores e chegam ao estado `qID`. Depois que o lexema completo é reconhecido, ele é consultado na tabela de símbolos. Caso esteja cadastrado como palavra reservada, seu token específico é utilizado. Caso contrário, permanece classificado como `ID`.

Esse procedimento segue a estratégia apresentada por Aho et al. [Aho et al. 2007, seção 3.4.2, págs. 132--133].

As palavras reservadas previstas no projeto são apresentadas na Tabela 4.

Tabela 4: Palavras reservadas definidas para a linguagem.

Lexema	Token
int	INT
float	FLOAT
char	CHAR
bool	BOOL
typedef	TYPDEF
struct	STRUCT
if	IF
else	ELSE
while	WHILE
readln	READLN
print	PRINT
break	BREAK
return	RETURN
true	TRUE
false	FALSE

2.1.9 Valores dos tokens

Os tipos pertencem a um `Enum` com `auto()`; o valor inteiro de cada membro está disponível em `tipo.value`. Esses valores pertencem à enumeração utilizada pelo projeto e são independentes dos números utilizados para identificar os estados do AFD.

Por exemplo, o número de um estado dentro do arquivo `AFD.json` serve para identificar uma posição do autômato, enquanto o valor de um token representa sua categoria dentro do restante do compilador.

A Tabela 5 apresentam os 47 valores utilizados pelo projeto.

Tabela 5: Tokens e valores inteiros utilizados pelo analisador.

Valor	Token	Exemplo
1	ID	x1
2	NUMINT	32
3	NUMFLOAT	12.50
4	LITERAL	"abc"
5	ASCII	'a'
6	COMMA	,
7	OPEN_BRACKETS	[
8	CLOSE_BRACKETS	]
9	LPARENT	(
10	RPARENT	)
11	OPEN_BRACES	{
12	CLOSE_BRACES	}
13	PLUS	+
14	MINUS	-
15	DECREMENT	--
16	INCREMENT	++
17	NEG	!
18	ASSIGN	=
19	EQUALS	==
20	LESS_THAN	<
21	LEQ	<=
22	GREATER_THAN	>
23	GEQ	>=
24	DIFF	!=
25	OR	
26	AND	&&
27	MULT	*
28	DIV	/
29	MOD	%
30	SEMICOLON	;
31	INT	int
32	FLOAT	float
33	CHAR	char
34	BOOL	bool
35	TYPEDDEF	typedef
36	STRUCT	struct
37	IF	if
38	ELSE	else
39	WHILE	while
40	READLN	readln
41	PRINT	print
42	BREAK	break
43	RETURN	return
44	TRUE	true
45	FALSE	false
46	EOF	fim da entrada
47	UNKNOWN	erro interno

2.1.10 Lexemas e atributos dos tokens

Além do seu tipo, cada token pode possuir informações relacionadas ao lexema reconhecido e à posição em que ele foi encontrado. [Aho et al. 2007, seção 3.1.3, págs. 112--113] apresentam o uso de atributos para associar informações adicionais aos tokens reconhecidos pelo analisador. Na implementação deste trabalho são utilizadas informações como:

- **tipo:** identifica a categoria do token;
- **lexema:** armazena a sequência de caracteres reconhecida;
- **linha:** indica a linha em que o lexema foi encontrado;
- **coluna:** indica sua posição inicial na linha;
- **indiceSimbolo:** associa cada token com lexema à entrada correspondente na tabela de símbolos; EOF não possui índice.

A Tabela 6 apresenta um exemplo para a entrada:

```
if(x1 <= 32) b = 10;
```

Tabela 6: Exemplos de tokens, lexemas e atributos.

Token	Lexema	Linha	Coluna	Índice
IF	if	1	1	4
LPARENT	(	1	3	18
ID	x1	1	4	40
LEQ	<=	1	7	30
NUMINT	32	1	10	41
RPARENT	)	1	12	19
ID	b	1	14	42
ASSIGN	=	1	16	27
NUMINT	10	1	18	43
SEMICOLON	;	1	20	39

Nesse exemplo, todos os tokens correspondentes a lexemas reconhecidos possuem uma referência para a tabela de símbolos. Identificadores e literais novos são inseridos durante a análise; palavras reservadas, operadores e símbolos de pontuação reutilizam entradas previamente carregadas.

2.1.11 Representação dos tokens na saída

Cada token é apresentado por seu nome na saída normal do programa; os valores inteiros não são impressos. Para a entrada da Tabela 6, a sequência produzida, com um token por linha, é:

```
IF
LPARENT
ID.x1
LEQ
NUMINT.32
RPARENT
ID.b
ASSIGN
NUMINT.10
SEMICOLON
EOF
```

Nos casos de identificadores e números, o lexema aparece junto ao nome do token. LITERAL e ASCII preservam o lexema no objeto, mas imprimem somente o nome da categoria.

2.1.12 AFD para reconhecimento dos tokens

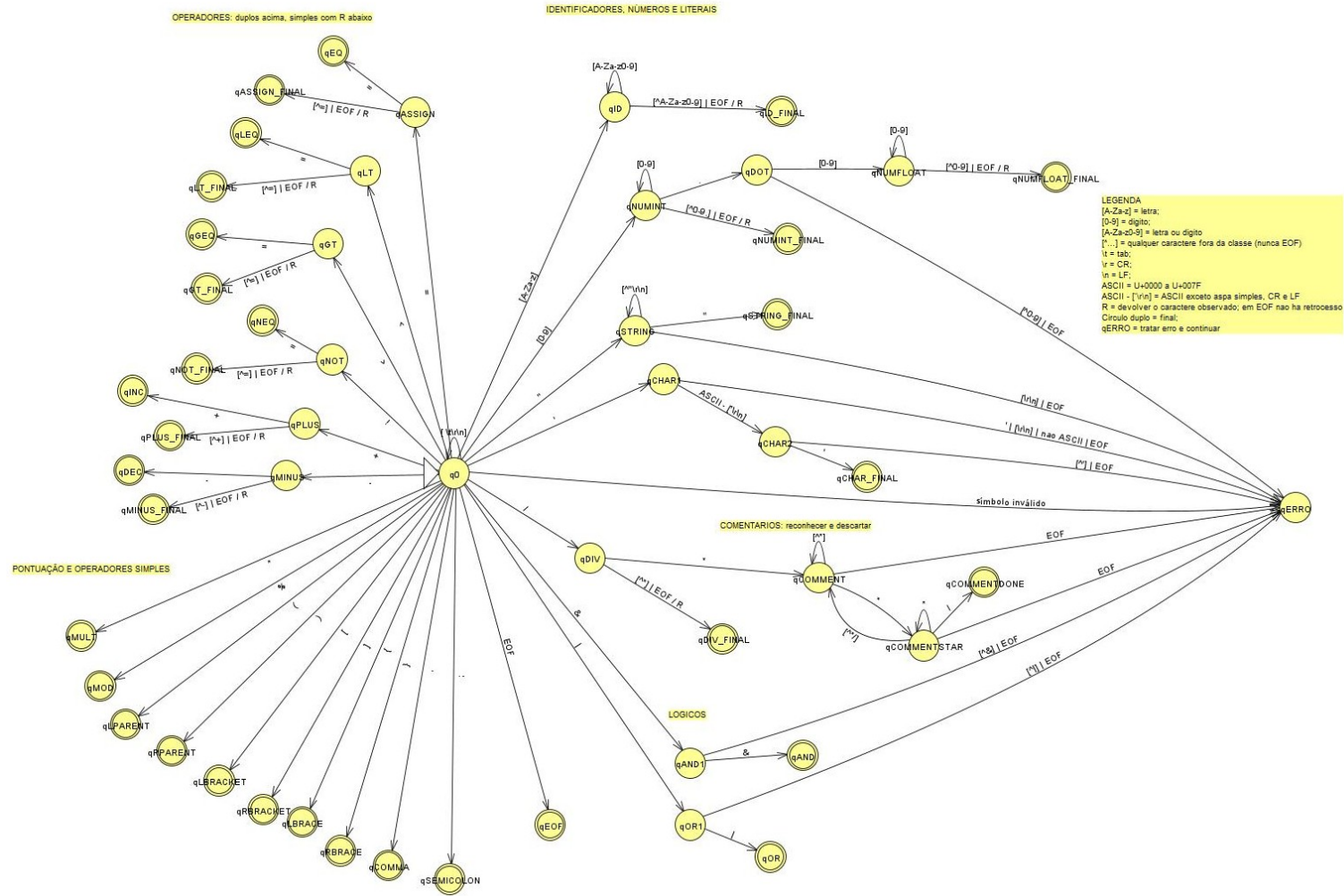


Figura 1: AFD projetado para o analisador léxico de C++. Estados com círculo duplo indicam aceite; R indica a devolução do caractere observado.

A Figura 1 reúne em um AFD os caminhos de identificadores, números, literais, operadores (inclusive lógicos), pontuação e comentários, agrupados para facilitar a leitura. Círculos duplos indicam aceite. O reconhecimento por autômatos segue Appel e Ginsburg [Appel e Ginsburg 1998, seção 2.3, págs. 21--23] e [Aho et al. 2007, seção 3.4, págs. 128--136].

2.1.13 Relação entre o AFD projetado e a implementação

O diagrama da Figura 1 é a referência visual do AFD projetado. Ele explicita estados auxiliares de finalização, como os terminados em `_FINAL`, o estado `qEOF` e as arestas marcadas com `R`, que devolvem ao fluxo de entrada o caractere que pertence ao próximo token.

A configuração executável em `AFD.json` codifica a mesma estratégia de reconhecimento com uma representação operacional mais compacta: estados como `qID`, `qNUMINT` e `qASSIGN` já pertencem ao conjunto de aceitação; o reconhecedor memoriza o último aceite e restaura o marco do leitor quando necessário; e o EOF é produzido diretamente pelo controle da entrada. Por isso, o desenho e o JSON devem ser comparados pelo comportamento léxico, e não por uma correspondência individual entre nomes e quantidade de estados. O JSON executado possui 41 estados e 31 finais; cada estado tem número e nome. O estado inicial é definido por:

```
"initial_state": 0
```

que corresponde ao estado `q0`. Os estados considerados de aceitação são armazenados separadamente em:

```
"final_states": [...]
```

A classe `AFD` carrega essa informação e verifica se o estado atual é final através do método:

```
def ehEstadoFinal(self):  
    return self.estadoAtual in self.estadosFinais
```

No JSON, `qID`, `qNUMINT`, `qNUMFLOAT` e `qASSIGN` pertencem diretamente ao conjunto de finais. O reconhecedor memoriza o último aceite para decidir a fronteira pelo maior lexema e restaura o cursor quando o caractere observado pertence ao token seguinte.

As transições também são armazenadas no JSON. Cada uma delas possui:

- o estado de origem;
- o estado de destino;
- a condição que deve ser satisfeita pelo caractere lido.

Por exemplo:

```
{"from": 0, "to": 1, "read": "letra"}
```

representa a transição:

```
q0 -- letra --> qID
```

Já:

```
{"from": 2, "to": 3, "read": "."}
```

representa a passagem de um número inteiro para o estado que aguarda a parte decimal.

O método `transicionar()` percorre as transições disponíveis para o estado atual e utiliza `_avaliarCondicao()` para verificar qual delas aceita o caractere recebido.

Quando existe uma lista de transições para o estado atual e nenhuma condição é satisfeita, o método move o autômato para o estado cujo nome é `qERRO` e retorna falso. Se o estado nem possui lista de transições, retorna falso sem mudá-lo. Em ambos os casos, o reconhecedor consulta o último aceite. Na configuração atual, o estado `qERRO` tem número 40:

Assim, o arquivo `AFD.json` fornece a estrutura operacional utilizada diretamente pelo programa, enquanto a Figura 1 registra o AFD de referência solicitado para o projeto. Os rótulos de EOF do JSON não são avaliados como eventos de fim de arquivo: o reconhecedor produz EOF diretamente, como explicado a seguir.

2.1.14 Fim do arquivo

O fim do arquivo é representado pelo token EOF, cujo valor inteiro é 46. Esse token possui uma função de controle e informa às demais partes do programa que todos os caracteres da entrada já foram processados.

Diferentemente dos demais lexemas, o EOF não corresponde a uma sequência escrita pelo usuário no arquivo fonte. Ele é produzido quando o analisador identifica que a entrada chegou ao fim.

2.2 Implementação

2.2.1 Entrada, saída e arquitetura

A Tabela 7 apresenta as classes e responsabilidades do analisador léxico, que são detalhadas nas seções seguintes.

Tabela 7: Classes e responsabilidades no Lexer.

Arquivo/classe	Responsabilidade efetivamente implementada
<code>main.py</code> , <code>src/Lexer.py</code>	Recebe argumentos e apresenta resultados; Valida o arquivo, recria tabela/erros e monta os colaboradores.
<code>src/core/LeitorTexto.py</code>	Carrega UTF-8 em memória; observa/avança caracteres e salva/restaura posição, linha e coluna.
<code>src/AFD.py</code> e <code>AFD.json</code>	Carregam estados, finais e listas de transições; avaliam condições por literais e predicados.
<code>src/core/Reconhecedor.py</code>	Encontra o maior lexema, descarta comentários iterativamente e agrupa construções incompletas para diagnóstico.
<code>src/core/MotorLexer.py</code>	Solicita tokens até EOF, consulta todos os lexemas na tabela única e encaminha erros ao gerenciador.
<code>src/TabelaSimbolos.py</code> , <code>GerenciadorErros.py</code>	Mantêm o dicionário de símbolos e a lista de diagnósticos.
<code>src/models/</code> e <code>TiposToken.py</code>	Definem registros de token, símbolo, erro e a enumeração dos tipos.

A Figura 2 apresenta o diagrama de classes do analisador léxico, com os módulos e classes do projeto. As setas indicam dependências entre classes. A interface de execução estão na Tabela 7.

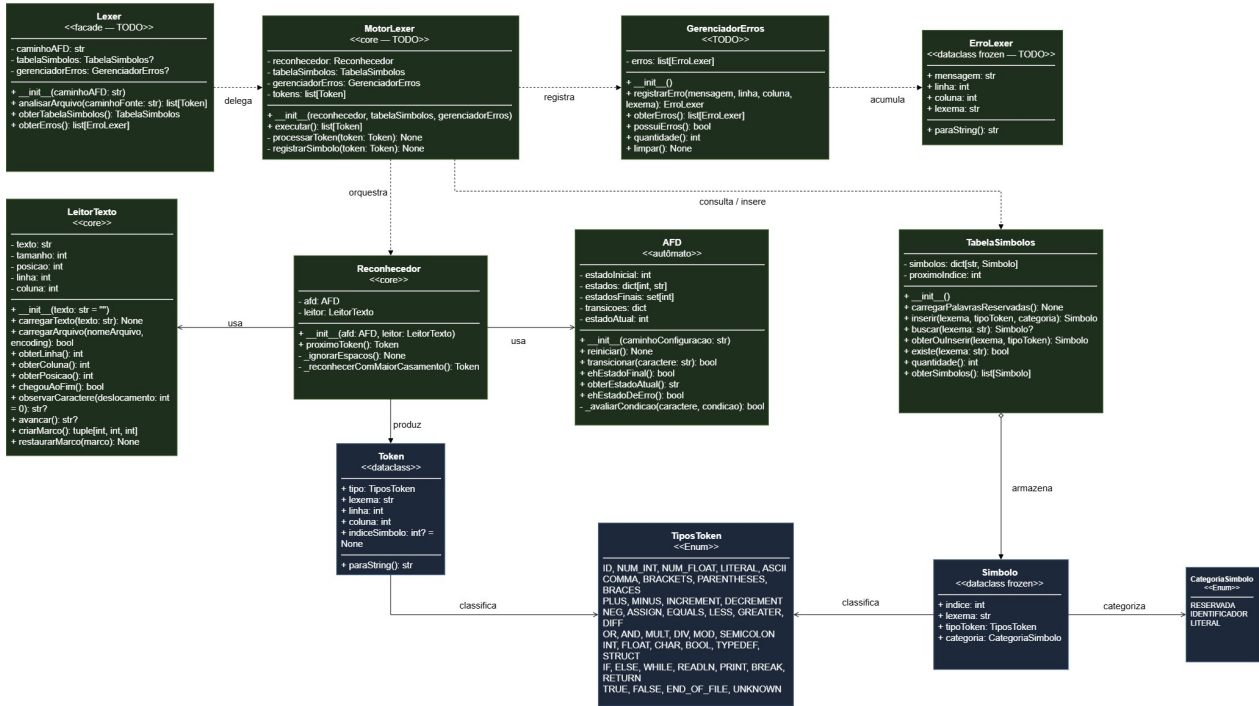


Figura 2: Diagrama de classes do analisador léxico.

A fachada cria uma tabela de símbolos e um gerenciador de erros por análise, enquanto reconhecedor e motor recebem somente as dependências necessárias. **Simbolo** é imutável; **Token** recebe o índice resolvido antes de ser devolvido. Essa divisão mantém leitura, reconhecimento, armazenamento e apresentação em componentes distintos.

As regras, exemplos e justificativas de cada classe de token estão na Subseção 2.1.

2.2.2 Algoritmo de reconhecimento e retrocesso

Estruturas e métodos. A configuração de estados, finais e transições e o método `ehEstadoFinal()` estão descritos na Subseção 2.1.13. Em memória, AFD usa dicionário de estados, conjunto de finais e listas de transições por origem. `reiniciar()` volta a q0; `_avaliarCondicao()` interpreta predicados conhecidos, sem executar expressões regulares arbitrárias do JSON. Essa representação segue [Appel e Ginsburg 1998, seção 2.3, págs. 21–23].

Algoritmo. `Reconhecedor.proximoToken()` implementa o maior lexema com memória do último estado final, como descrito por [Appel e Ginsburg 1998, seção 2.3, pág. 23] e [Aho et al. 2007, seção 3.4, págs. 128–136]:

1. Reinicia o AFD, ignora separadores e retorna EOF se a entrada acabou.
2. Salva o início e avança enquanto há transição válida. A cada estado final, salva o lexema e o marco de posição, linha e coluna.
3. Se encontrou um final, restaura seu marco e classifica pelo mapa `ESTADO_PARA_TOKEN`.
4. Se reconheceu `qCOMMENTDONE`, descarta o comentário e reinicia o laço, sem recursão.
5. Se string, caractere, decimal ou comentário ficaram incompletos, agrupa o lexema inválido e retorna UNKNOWN com mensagem específica.
6. Nos demais casos sem final, restaura o início, consome um caractere e retorna UNKNOWN, garantindo progresso.

`MotorLexex.executar()` repete a solicitação até EOF; `processarToken()` encaminha UNKNOWN ao gerenciador e consulta cada lexema válido na tabela, inclusive reservadas, operadores e delimi-

tadores. `LeitorTexto.criarMarco()` e `restaurarMarco()` salvam/restauram uma tupla de três inteiros. O armazenamento integral substitui o esquema de dois buffers de [Aho et al. 2007, seção 3.2, págs. 115--117]; favorece a simplicidade, com custo de memória proporcional ao arquivo.

Fronteira do token. Em `=x`, o reconhecedor salva o aceite de `=` e retorna `ASSIGN`, deixando `x` para a próxima chamada. A relação entre o algoritmo, a codificação em JSON e o desenho está na Subseção 2.1.13. [Disciplina de Compiladores s.d., pág. 1].

2.3 Tabela de símbolos

A classe `TabelaSimbolos` mantém um único `dict[str, Simbolo]` compartilhado durante cada análise. A chave é o lexema completo; o valor imutável reúne índice, lexema, tipo de token e categoria. Dicionários fornecem busca associativa, conforme a implementação por hash discutida por [Appel e Ginsburg 1998, seção 5.1, págs. 106--109]. A consulta prévia das palavras reservadas segue [Aho et al. 2007, seção 3.4.2, págs. 132--133].

2.3.1 Categorias e conteúdo inicial

`CategoriaSimbolo` diferencia quatro grupos: `RESERVADA`, `IDENTIFICADOR`, `LITERAL` e `FIXO`. O último reúne operadores e delimitadores; ele permite cumprir a exigência de pesquisar toda string reconhecida antes de devolver o token [Disciplina de Compiladores s.d., pág. 3].

O construtor chama `carregarTokensIniciais()`. Primeiro insere as 15 palavras reservadas da Tabela 4; depois insere os 25 lexemas fixos, como `+`, `<=`, parênteses e `;`. Cada entrada recebe um índice crescente e estável na análise.

2.3.2 Operações e algoritmo

Tabela 8: Interface de `TabelaSimbolos`.

Método	Contrato
<code>carregarTokensIniciais()</code>	Insere reservadas e lexemas fixos com seus tipos e categorias.
<code>inserir(lexema, tipoToken, categoria)</code>	Reutiliza a entrada existente ou cria uma nova com o próximo índice.
<code>buscar(lexema)</code>	Retorna o <code>Simbolo</code> associado ou <code>None</code> .
<code>obterOuInserir(lexema, tipoToken)</code>	Consulta todo lexema; se novo, classifica-o como identificador, literal ou fixo e insere.
<code>existe(lexema)</code>	Informa se o lexema já está armazenado.
<code>remover(lexema)</code>	Remove uma entrada e informa se ela existia; não renumera as demais.
<code>quantidade()</code>	Retorna o número total de entradas.

O motor chama `obterOuInserir()` para todo token válido, com exceção de `EOF`, que não possui lexema. Se a entrada já existe, seu tipo prevalece: assim, o AFD reconhece `if` inicialmente pelo padrão de `ID`, mas a tabela o resolve como `TiposToken.IF`. Identificadores e literais novos são cadastrados, e o índice encontrado é copiado para `Token.indiceSimbolo`. Operadores e delimitadores encontram suas entradas previamente carregadas. Essa referência por índice corresponde ao atributo de token descrito por [Aho et al. 2007, seção 3.1.3, págs. 112--113].

Ocorrências repetidas reutilizam o mesmo registro. Em uma análise de `x x "a" "a"`, os dois `IDs` compartilham um índice, as duas strings compartilham outro e a tabela termina com 42 entradas: 40 iniciais e duas dinâmicas. Uma nova chamada à fachada recria a tabela, impedindo vazamento de símbolos entre arquivos. Escopos e declarações pertencem às fases sintática e semântica, não à classificação léxica [Aho et al. 2007, seção 2.7, pág. 86].

2.4 Gerenciador de erros léxicos

O gerenciador acumula diagnósticos sem interromper a análise. A recuperação por descarte controlado se apoia em [Aho et al. 2007, seção 3.1.4, págs. 113--114] e [Appel e Ginsburg 1998, seção 2.2, Fig. 2.2, pág. 20].

2.4.1 Estrutura e interface

`GerenciadorErros.erros` é uma lista de registros imutáveis `ErroLexer`, cada um com mensagem, linha, coluna e lexema. A contagem deriva do tamanho da lista, sem estado redundante.

Tabela 9: Interface do gerenciador de erros.

Método	Contrato
<code>registrarErro(mensagem, linha, coluna, lexema)</code>	Cria, armazena e retorna um diagnóstico.
<code>registrarCaractereInvalido(caractere, linha, coluna)</code>	Atalho para diagnosticar um caractere inválido.
<code>possuiErros()</code>	Informa se algum erro foi registrado.
<code>quantidade()</code>	Retorna a quantidade de erros.
<code>obterErros()</code>	Retorna uma cópia da lista.
<code>limpar()</code>	Remove os diagnósticos anteriores.

2.4.2 Detecção, recuperação e apresentação

O reconhecedor associa uma mensagem ao marcador UNKNOWN. Para um caractere isolado inválido, restaura o marco inicial, consome exatamente uma unidade e usa “Token não reconhecido”. Para construções iniciadas que não podem terminar validamente, consome o fragmento como uma unidade e informa: “Número decimal incompleto”, “Literal de string não terminado”, “Literal de caractere inválido ou não terminado” ou “Comentário não terminado”.

O motor registra o erro, omite UNKNOWN da sequência e solicita o token seguinte. Quebras de linha não são consumidas junto com strings ou caracteres malformados, preservando a posição da próxima tentativa. Linhas e colunas começam em 1 e CRLF conta como uma única mudança de linha. Para "aberto na linha 3, coluna 8, o formato apresentado é:

```
Erro léxico na linha 3, coluna 8: Literal de string não terminado.  
Lexema: '"aberto'
```

`main.py` imprime somente tokens válidos, os diagnósticos e retorna código 1 quando há erro léxico. O processo continua até EOF, permitindo relatar múltiplos problemas em uma única execução.

2.5 Resultados experimentais

A verificação automatizada executa a fachada real e compara tokens, diagnósticos e posições com expectativas explícitas. O programa empregado no teste principal é apresentado integralmente a seguir. Ele reúne palavras reservadas, identificadores, literais, todos os grupos de operadores, delimitadores, comentário multilinha e três erros léxicos intencionais.

Programa 1: Programa utilizado no teste principal.

```
1  /* Teste */
2  typedef struct {
3      int codigo;
4      float saldo;
5      char categoria;
6      bool ativo;
7  } Conta;
8
9  int calcular(int a, int b) {
10     int resultado;
11     resultado = (a + b) * 2 / 1 % 3 - 1;
12     return resultado;
13 }
14
15 int main() {
16     Conta contas[10];
17     int indice;
18     float valor;
19     char letra;
20     bool valido;
21     int x, y;
22
23     x+++++y;
24     x++ + ++y;
25
26     indice = 0;
27     valor = 12.50;
28     letra = 'A';
29     valido = true;
30
31     /* O comentario pode ocupar
32        mais de uma linha. */
33     while (indice < 10 && valido == true) {
34         readln(valor);
35
36         if (valor >= 100.0 || valor != 0.0) {
37             print("valor valido");
38             indice++;
39         } else {
40             if (valor <= 0.0 && indice > 0) {
41                 indice--;
42             }
43
44             valido = !false;
45             break;
46         }
47
48         /*erros*/
49         float valor@base = 10.5; /* O caractere @ nao e reconhecido pelo lexer. */
50         print("valor valido");
51         valor = 12.50.8;
52     }
53
54     return calcular(indice, 2);
55 }
```

A execução percorreu todo o programa, produziu 192 tokens válidos, incluindo EOF, e terminou com código 1 devido aos três erros esperados. O caractere @ foi diagnosticado na linha 49, coluna 20; a string sem fechamento, na linha 50, coluna 15; e o segundo ponto da construção 12.50.8, na linha 51, coluna 22. Mesmo após cada ocorrência, o analisador continuou até o fim da entrada. Os casos exercitam maior lexema e recuperação, discutidos por [Appel e Ginsburg 1998, seções 2.2--2.3, págs. 20--23] e [Aho et al. 2007, seção 3.1.4, pág. 114].

Tabela 10: Entradas e saídas verificadas; erros contados separadamente.

Entrada	Sequência de saída	Erros
if ifx	IF ID.ifx EOF	0
32 12.50	NUMINT.32 NUMFLOAT.12.50 EOF	0
= ==	ASSIGN EQUALS EOF	0
' '	ASCII EOF	0
" "	LITERAL EOF	0
/* ok */x	ID.x EOF	0
' '	EOF	1
' '	EOF	2
U+0661 U+0662	EOF	2
@\nx	ID.x EOF	1
12.	EOF	1
"abc	EOF	1
/* aberto	EOF	1

Todos os casos da tabela corresponderam às expectativas. Comentário, string, caractere e decimal incompletos produzem um diagnóstico específico por construção (três aspas simples formam duas construções inválidas). O caso com @ comprova a continuidade e as posições (1,1) para o erro e (2,1) para o ID seguinte. EOF foi verificado em toda entrada.

Também passaram as verificações de identificador e inteiro com 10.000 caracteres, reutilização de índices, 1.500 comentários consecutivos sem recursão e o exemplo do enunciado.

2.6 Considerações finais

A entrega implementa reconhecimento por maior lexema, classificação de reservadas, consulta de todo lexema válido em uma tabela única, EOF e continuidade após erros. Comentários completos são descartados de forma iterativa; comentários e literais incompletos recebem diagnósticos específicos sem impedir o reconhecimento posterior. Esses resultados concretizam as técnicas de Appel e Ginsburg [Appel e Ginsburg 1998, seções 2.2--2.3, págs. 20--23] e [Aho et al. 2007, seções 3.1.3--3.1.4, págs. 112--114] dentro do escopo léxico solicitado.

As próximas entregas podem consumir os tipos, posições e índices estáveis aqui definidos, preservando a separação entre as fases do compilador.

2.7 Referências Bibliográficas

- [Aho et al. 2007] AHO, A. V. et al. *Compilers: Principles, Techniques, and Tools*. 2. ed. Boston: Addison-Wesley, 2007.
- [Appel e Ginsburg 1998] APPEL, A. W.; GINSBURG, M. *Modern Compiler Implementation in C*. Cambridge: Cambridge University Press, 1998.
- [Disciplina de Compiladores s.d.] Disciplina de Compiladores. *Projeto e implementação de compilador: parte 1*. s.d. Enunciado fornecido pela disciplina.
- [ISO/IEC JTC 1/SC 22/WG14 2024] ISO/IEC JTC 1/SC 22/WG14. *Information technology --- Programming languages --- C: N3220 working draft*. 2024. Disponível em: <<https://www.open-std.org/jtc1/sc22/wg14/www/docs/n3220.pdf>>. Acesso em: 12 set. 2026.